

Name: R Priya Dharshini

Roll: CH.SC.U4CYS25026

ACTIVITY – 3

Lab Activity 3

1. Create an array 'A' of 1000 elements and perform quick sort (as discussed in class room). Measure the time complexity and execution time.
2. Perform merge sort for the same array 'A' of 1000 elements. Measure the time complexity and execution time.
3. Apply Quick Sort for 4 array segments, A1 = A[0:249], A2 = A[250:499], A3 = A[500:749], A4 = A[750:999]. Sorted segments must be in respective locations, A[0:249] = A1, A[250:499] = A2, A[500:749] = A3, A[750:999] = A4. Follow merge-sort combine to merge the sorted segments. Measure the time complexity and execution time.
4. Improve the time complexity and execution time of above algorithms wherever applicable without increasing space complexity. (Refer Online and Add Your Understanding)

Activity (At Home):

1. Implement and analyze the below algorithms and evaluate its time complexity and execution time.
 - a. Heap Sort
 - b. ~~Bogo~~ Sort
 - c. Comb Sort
 - d. Selection Sort
 - e. Insertion Sort

Compare their merits and de-merits in terms of time complexity and execution time.

ANSWER 1:

Program

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
#include<math.h>

void swap(int *a,int *b)
{
    int temp=*a;
    *a=*b;
    *b=temp;
}

int partition(int arr[],int low,int high)
{
    int pivot=arr[high];
    int i=low-1;

    for(int j=low;j<high;j++)
```

```

    {
        if(arr[j]<=pivot)
        {
            i++;
            swap(&arr[i],&arr[j]);
        }
    }

    swap(&arr[i+1],&arr[high]);
    return i+1;
}

void quickSort(int arr[],int low,int high)
{
    if(low<high)
    {
        int pi=partition(arr,low,high);

        quickSort(arr,low,pi-1);
        quickSort(arr,pi+1,high);
    }
}

int main()
{
    srand(time(NULL));

    int a[1000];

    for(int i=0;i<1000;i++)
        a[i]=rand()%1000;

    printf("Before Sorting (First 20 Elements):\n");
    for(int i=0;i<20;i++)
        printf("%d ",a[i]);

    clock_t start,end;

    start=clock();
    quickSort(a,0,999);
    end=clock();

    printf("\n\nAfter Sorting (First 20 Elements):\n");
    for(int i=0;i<20;i++)
        printf("%d ",a[i]);

    double timec=((double)(end-start))/CLOCKS_PER_SEC;

    int a1[4000];

    for(int i=0;i<4000;i++)
        a1[i]=rand()%4000;

```

```

start=clock();
quickSort(a1,0,3999);
end=clock();

double timc1=((double)(end-start))/CLOCKS_PER_SEC;

printf("\n\nExecution Time (1000 Elements) : %lf seconds",timec);
printf("\nExecution Time (4000 Elements) : %lf seconds",timc1);

printf("\nRatio (Experimental) : %lf",timc1/timec);

double t1=1000*(log(1000)/log(2));
double t2=4000*(log(4000)/log(2));

printf("\nRatio (Theoretical) : %lf",t2/t1);

printf("\n\nTime Complexity:");
printf("\nBest Case   : O(n log n)");
printf("\nAverage Case : O(n log n)");
printf("\nWorst Case  : O(n^2)");

return 0;
}

```

Output

```

Before Sorting (First 20 Elements):
83 639 765 683 725 888 576 333 558 921 348 747 535 382 695 418 743 356 576 29

After Sorting (First 20 Elements):
0 2 5 6 8 9 9 9 11 12 12 13 18 18 18 18 19 19 19 20

Execution Time (1000 Elements) : 0.000106 seconds
Execution Time (4000 Elements) : 0.000414 seconds
Ratio (Experimental) : 3.905660
Ratio (Theoretical) : 4.802747

Time Complexity:
Best Case   : O(n log n)
Average Case : O(n log n)
Worst Case  : O(n^2)

=== Code Execution Successful ===

```

ANSWER 2:

Program

```

#include<stdio.h>
#include<stdlib.h>
#include<time.h>
#include<math.h>

```

```

void merge(int arr[],int low,int mid,int high)
{
    int temp[4000];
    int i=low,j=mid+1,k=low;

    while(i<=mid && j<=high)
    {
        if(arr[i]<=arr[j])
            temp[k++]=arr[i++];
        else
            temp[k++]=arr[j++];
    }

    while(i<=mid)
        temp[k++]=arr[i++];

    while(j<=high)
        temp[k++]=arr[j++];

    for(i=low;i<=high;i++)
        arr[i]=temp[i];
}

```

```

void mergeSort(int arr[],int low,int high)
{
    if(low<high)
    {
        int mid=(low+high)/2;

        mergeSort(arr,low,mid);
        mergeSort(arr,mid+1,high);

        merge(arr,low,mid,high);
    }
}

```

```

int main()
{
    srand(time(NULL));

    int a[1000];

    for(int i=0;i<1000;i++)
        a[i]=rand()%1000;

    printf("Before Sorting (First 20 Elements):\n");
    for(int i=0;i<20;i++)
        printf("%d ",a[i]);

    clock_t start,end;

    start=clock();

```

```

mergeSort(a,0,999);
end=clock();

printf("\n\nAfter Sorting (First 20 Elements):\n");
for(int i=0;i<20;i++)
    printf("%d ",a[i]);

double timc=((double)(end-start))/CLOCKS_PER_SEC;

int a1[4000];

for(int i=0;i<4000;i++)
    a1[i]=rand()%4000;

start=clock();
mergeSort(a1,0,3999);
end=clock();

double timc1=((double)(end-start))/CLOCKS_PER_SEC;

printf("\n\nExecution Time (1000 Elements) : %lf seconds",timc);
printf("\n\nExecution Time (4000 Elements) : %lf seconds",timc1);

printf("\n\nRatio (Experimental) : %lf",timc1/timc);

double t1=1000*(log(1000)/log(2));
double t2=4000*(log(4000)/log(2));

printf("\n\nRatio (Theoretical) : %lf",t2/t1);

printf("\n\nTime Complexity:");
printf("\nBest Case   : O(n log n)");
printf("\nAverage Case : O(n log n)");
printf("\nWorst Case  : O(n log n)");

return 0;
}

```

Output

```
Before Sorting (First 20 Elements):  
729 164 789 622 879 765 612 450 273 365 422 308 975 910 321 694 612 603 918 502
```

```
After Sorting (First 20 Elements):  
0 1 3 3 4 5 6 6 7 8 8 8 9 9 9 10 11 11 11 12
```

```
Execution Time (1000 Elements) : 0.000149 seconds  
Execution Time (4000 Elements) : 0.000665 seconds  
Ratio (Experimental) : 4.463087  
Ratio (Theoretical) : 4.802747
```

```
Time Complexity:  
Best Case :  $O(n \log n)$   
Average Case :  $O(n \log n)$   
Worst Case :  $O(n \log n)$ 
```

```
=== Code Execution Successful ===
```

ANSWER 3:

Program

```
#include<stdio.h>  
#include<stdlib.h>  
#include<time.h>  
  
void swap(int *a,int *b)  
{  
    int temp=*a;  
    *a=*b;  
    *b=temp;  
}  
  
int partition(int arr[],int low,int high)  
{  
    int pivot=arr[high];  
    int i=low-1;  
  
    for(int j=low;j<high;j++)  
    {  
        if(arr[j]<=pivot)  
        {  
            i++;  
            swap(&arr[i],&arr[j]);  
        }  
    }  
  
    swap(&arr[i+1],&arr[high]);  
    return i+1;  
}
```

```

void quickSort(int arr[],int low,int high)
{
    if(low<high)
    {
        int pi=partition(arr,low,high);

        quickSort(arr,low,pi-1);
        quickSort(arr,pi+1,high);
    }
}

void merge(int arr[],int low,int mid,int high)
{
    int temp[1000];
    int i=low,j=mid+1,k=low;

    while(i<=mid && j<=high)
    {
        if(arr[i]<=arr[j])
            temp[k++]=arr[i++];
        else
            temp[k++]=arr[j++];
    }

    while(i<=mid)
        temp[k++]=arr[i++];

    while(j<=high)
        temp[k++]=arr[j++];

    for(i=low;i<=high;i++)
        arr[i]=temp[i];
}

int main()
{
    srand(time(NULL));

    int a[1000];

    for(int i=0;i<1000;i++)
        a[i]=rand()%1000;

    printf("Before Sorting (First 20 Elements):\n");
    for(int i=0;i<20;i++)
        printf("%d ",a[i]);

    clock_t start,end;

    start=clock();

    quickSort(a,0,249);
    quickSort(a,250,499);

```

```

quickSort(a,500,749);
quickSort(a,750,999);

merge(a,0,249,499);
merge(a,500,749,999);
merge(a,0,499,999);

end=clock();

printf("\n\nAfter Sorting (First 20 Elements):\n");
for(int i=0;i<20;i++)
    printf("%d ",a[i]);

double time=((double)(end-start))/CLOCKS_PER_SEC;

printf("\n\nExecution Time : %lf seconds",time);

printf("\n\nTime Complexity:");
printf("\nQuick Sort : O(n log n)");
printf("\nMerge      : O(n)");
printf("\nOverall    : O(n log n)");

return 0;
}

```

Output

```

Before Sorting (First 20 Elements):
310 853 467 501 544 657 925 158 915 34 310 672 334 284 83 42 961 113 930 250

After Sorting (First 20 Elements):
0 1 2 3 4 4 5 5 5 5 6 7 8 9 9 10 10 11 11 14

Execution Time : 0.000099 seconds

Time Complexity:
Quick Sort : O(n log n)
Merge      : O(n)
Overall    : O(n log n)

=== Code Execution Successful ===

```

ANSWER 4:

- ❖ Choose a better pivot (Random Pivot or Median-of-Three Pivot) to avoid highly unbalanced partitions.
- ❖ Use Insertion Sort for very small sub-arrays (less than 10–20 elements).
- ❖ Avoid unnecessary recursive calls by checking the partition size.
- ❖ Apply Tail Recursion Optimization to reduce recursive overhead.

❖ Sort the smaller partition first, which reduces the recursion depth.

ANSWER for Activity(At Home-1):

Program

```
/* Sorting Program
 * Algorithms:
 * 1. Heap Sort
 * 2. Bogo Sort (uses first 8 elements only)
 * 3. Comb Sort
 * 4. Selection Sort
 * 5. Insertion Sort
 */

#include<stdio.h>
#include<stdlib.h>
#include<time.h>

#define SIZE 1000
#define BOGO_SIZE 8

void swap(int *a,int *b){int t=*a;*a=*b;*b=t;}

void fillRandom(int a[],int n){
    for(int i=0;i<n;i++) a[i]=rand()%10000;
}

void printArray(int a[],int n){
    for(int i=0;i<n;i++){
        printf("%5d ",a[i]);
        if((i+1)%20==0) printf("\n");
    }
    printf("\n");
}

/* Heap Sort */
void heapify(int a[],int n,int i){
    int largest=i,l=2*i+1,r=2*i+2;
    if(l<n&& a[l]>a[largest]) largest=l;
    if(r<n&& a[r]>a[largest]) largest=r;
    if(largest!=i){
        swap(&a[i],&a[largest]);
        heapify(a,n,largest);
    }
}

void heapSort(int a[],int n){
    for(int i=n/2-1;i>=0;i--) heapify(a,n,i);
    for(int i=n-1;i>0;i--){
        swap(&a[0],&a[i]);
        heapify(a,i,0);
    }
}
```

```

/* Selection Sort */
void selectionSort(int a[],int n){
    for(int i=0;i<n-1;i++){
        int min=i;
        for(int j=i+1;j<n;j++){
            if(a[j]<a[min]) min=j;
            swap(&a[i],&a[min]);
        }
    }
}

/* Insertion Sort */
void insertionSort(int a[],int n){
    for(int i=1;i<n;i++){
        int key=a[i],j=i-1;
        while(j>=0&& a[j]>key){
            a[j+1]=a[j];
            j--;
        }
        a[j+1]=key;
    }
}

/* Comb Sort */
void combSort(int a[],int n){
    int gap=n;
    int swapped=1;
    while(gap!=1 || swapped){
        gap=(gap*10)/13;
        if(gap<1) gap=1;
        swapped=0;
        for(int i=0;i+gap<n;i++){
            if(a[i]>a[i+gap]){
                swap(&a[i],&a[i+gap]);
                swapped=1;
            }
        }
    }
}

/* Bogo Sort */
int sorted(int a[],int n){
    for(int i=1;i<n;i++){
        if(a[i-1]>a[i]) return 0;
    }
    return 1;
}
void shuffle(int a[],int n){
    for(int i=0;i<n;i++){
        int j=rand()%n;
        swap(&a[i],&a[j]);
    }
}
void bogoSort(int a[],int n){

```

```

while(!sorted(a,n))
    shuffle(a,n);
}

void complexity(int ch){
    printf("\nTime Complexity\n");
    switch(ch){
        case 1: printf("Best: O(n log n)\nAverage: O(n log n)\nWorst: O(n log n)\nSpace: O(1)\n"); break;
        case 2: printf("Best: O(n)\nAverage: O(n*n!)\nWorst: Infinite\nSpace: O(1)\n"); break;
        case 3: printf("Best: O(n log n)\nAverage: O(n^2)\nWorst: O(n^2)\nSpace: O(1)\n"); break;
        case 4: printf("Best: O(n^2)\nAverage: O(n^2)\nWorst: O(n^2)\nSpace: O(1)\n"); break;
        case 5: printf("Best: O(n)\nAverage: O(n^2)\nWorst: O(n^2)\nSpace: O(1)\n"); break;
    }
}

int main(){
    int a[SIZE],ch;
    srand(time(NULL));

    do{
        fillRandom(a,SIZE);

        printf("1. Heap Sort\n");
        printf("2. Bogo Sort (8 elements)\n");
        printf("3. Comb Sort\n");
        printf("4. Selection Sort\n");
        printf("5. Insertion Sort\n");
        printf("0. Exit\n");
        printf("Enter choice: ");
        scanf("%d",&ch);

        if(ch==0) break;

        printf("\nUnsorted Array (first 100 elements)\n");
        if(ch==2) printArray(a,BOGO_SIZE);
        else printArray(a,100);

        clock_t start=clock();

        switch(ch){
            case 1: heapSort(a,SIZE); break;
            case 2: bogoSort(a,BOGO_SIZE); break;
            case 3: combSort(a,SIZE); break;
            case 4: selectionSort(a,SIZE); break;
            case 5: insertionSort(a,SIZE); break;
            default: printf("Invalid Choice\n"); continue;
        }
    }
}

```

```
clock_t end=clock();

printf("\nSorted Array\n");
if(ch==2) printArray(a,BOGO_SIZE);
else printArray(a,100);

printf("\nExecution Time: %.6f seconds\n",
(double)(end-start)/CLOCKS_PER_SEC);

complexity(ch);

}while(1);

return 0;
}
```

Output

===== SORT MENU =====

1. Heap Sort
2. Bogo Sort (8 elements)
3. Comb Sort
4. Selection Sort
5. Insertion Sort
0. Exit

Enter choice: 1

Unsorted Array (first 20 elements)

3025	6645	3669	4270	3918	5232	949	1986	4851	3557	576	4759	5617	5734
9493	3627	3007	2050	516	116								

Sorted Array

0	0	1	8	9	28	91	96	98	103	116	122	124	129
	161	167	190	208	211	223							

Execution Time: 0.000137 seconds

Time Complexity

Best: $O(n \log n)$

Average: $O(n \log n)$

Worst: $O(n \log n)$

Space: $O(1)$

===== SORT MENU =====

1. Heap Sort
2. Bogo Sort (8 elements)
3. Comb Sort
4. Selection Sort
5. Insertion Sort
0. Exit

Enter choice: 2

Unsorted Array (first 20 elements)

6800 660 1854 2521 5999 9074 2375 2579

Sorted Array

660 1854 2375 2521 2579 5999 6800 9074

Execution Time: 0.001780 seconds

Time Complexity

Best: $O(n)$

Average: $O(n \cdot n!)$

Worst: Infinite

Space: $O(1)$

===== SORT MENU =====

1. Heap Sort
2. Bogo Sort (8 elements)
3. Comb Sort
4. Selection Sort
5. Insertion Sort
0. Exit

Enter choice: 3

Unsorted Array (first 20 elements)

3015	5871	5669	5336	8267	7401	6407	1397	5649	6045	7626	3746	9556	6327
5617	9414	4061	7469	1518	2093								

Sorted Array

9	11	11	35	45	46	56	57	61	78	82	86	103	106
126	138	153	155	168	171								

Execution Time: 0.000146 seconds

Time Complexity

Best: $O(n \log n)$

Average: $O(n^2)$

Worst: $O(n^2)$

Space: $O(1)$

===== SORT MENU =====

1. Heap Sort
2. Bogo Sort (8 elements)
3. Comb Sort
4. Selection Sort
5. Insertion Sort
0. Exit

Enter choice: 4

Unsorted Array (first 20 elements)

5737	3404	740	3776	1730	6915	2709	9309	7632	1732	7991	4011	375	1392
6560	1873	9596	5620	4908	5218								

Sorted Array

37	46	47	53	80	100	125	130	147	153	159	163	170	182
194	197	200	211	220	226								

Execution Time: 0.001414 seconds

Time Complexity

Best: $O(n^2)$

Average: $O(n^2)$

Worst: $O(n^2)$

Space: $O(1)$

```

===== SORT MENU =====
1. Heap Sort
2. Bogo Sort (8 elements)
3. Comb Sort
4. Selection Sort
5. Insertion Sort
0. Exit
Enter choice: 5

Unsorted Array (first 20 elements)
5645  443  5784  544  8366  366  5165  845  8901  2228  3821  8546  8843  7837
7927  7745  9692  4430  3291  226

Sorted Array
12    20    26    44    53    58    59    60    67    68    75    78    79    81
118   119   124   126   133   134

Execution Time: 0.000747 seconds

Time Complexity
Best: O(n)
Average: O(n^2)
Worst: O(n^2)
Space: O(1)

```

Comparison of Sorting Algorithms

Comparison of merits and de-merits in terms of time complexity and execution time.

Algorithm	Time Complexity	Execution Time	Merits	Demerits
Heap Sort	Best: $O(n \log n)$ Average: $O(n \log n)$ Worst: $O(n \log n)$	Fast and consistent for large datasets.	Guaranteed $O(n \log n)$, in-place, suitable for large datasets.	Not stable; usually slower than Quick Sort in practice.
Bogo Sort	Best: $O(n)$ Average: $O(n \times n!)$ Worst: Infinite/Unbounded	Extremely slow; practical only for 5–8 elements.	Very simple; useful for educational purposes.	Impractical for real applications due to enormous execution time.
Comb Sort	Best: $O(n \log n)$ Average: $\approx O(n^2)$ Worst: $O(n^2)$	Faster than Bubble Sort but slower than Heap Sort.	Simple; improves Bubble Sort by eliminating turtles.	Not stable; still inefficient for very large datasets.

Selection Sort	Best: $O(n^2)$ Average: $O(n^2)$ Worst: $O(n^2)$	Slow for large datasets.	Simple, minimum swaps, $O(1)$ extra space.	Always $O(n^2)$; not suitable for large datasets.
Insertion Sort	Best: $O(n)$ Average: $O(n^2)$ Worst: $O(n^2)$	Very fast for small/nearly sorted arrays.	Stable, simple, efficient for small datasets.	Slow for large random datasets.

Conclusion

Heap Sort is the best choice for large datasets because it guarantees $O(n \log n)$ time in all cases. Insertion Sort is efficient for small or nearly sorted arrays. Selection Sort is simple but inefficient for large data. Comb Sort improves over Bubble Sort but remains quadratic in the worst case. Bogo Sort is intended only for educational demonstrations due to its extremely poor execution time.